

DOCUMENTACIÓN DEL PROYECTO

SISTEMA DE GESTIÓN DEPORTIVA

CLUB VILLA DEL PARQUE II



Institución: IFTS N°16

Carrera: Analista de sistemas

Materia: Técnicas de programación y Base de datos

Profesor: Ingrid García y Gustavo E. Escandell

Alumno: Gastón Toranzo

Trabajo Práctico Final

Sistema de Gestión Deportiva – Club Villa del Parque II

Fecha: 18/06/26

1. INTRODUCCIÓN

La informatización de procesos administrativos constituye una necesidad fundamental para cualquier organización que requiera gestionar grandes volúmenes de información de manera eficiente.

Los clubes deportivos administran diariamente datos relacionados con socios, actividades deportivas, inscripciones, pagos, invitados y estadísticas de gestión. Cuando estos procesos son realizados manualmente, suelen producirse errores de registro, pérdida de información, demoras administrativas y dificultades para obtener reportes confiables.

Con el objetivo de solucionar esta problemática se desarrolló el Sistema de Gestión Deportiva Club Villa del Parque II, una aplicación de escritorio diseñada para centralizar y administrar toda la información operativa del club.

El sistema fue desarrollado utilizando Python como lenguaje principal de programación, SQL Server como gestor de base de datos relacional y Tkinter para la construcción de la interfaz gráfica de usuario.

La solución implementada permite gestionar socios, deportes, inscripciones, pagos, invitados y reportes administrativos, brindando una herramienta integral para la administración del club.

2. FUNDAMENTACIÓN DEL PROYECTO

La gestión manual de la información presenta múltiples inconvenientes:

- Duplicación de datos.
- Errores de carga.
- Falta de control sobre los pagos.
- Dificultad para obtener estadísticas.
- Imposibilidad de realizar consultas rápidas.
- Escasa trazabilidad de la información.

La implementación de un sistema informático permite:

- Centralizar la información.
- Mantener integridad de los datos.
- Mejorar la velocidad de acceso.
- Facilitar la toma de decisiones.
- Reducir errores administrativos.
- Automatizar procesos repetitivos.

Por estas razones se consideró adecuado desarrollar una solución informática específica para las necesidades del Club Villa del Parque II.

3. OBJETIVOS

3.1 Objetivo General

Desarrollar un sistema de gestión deportiva que permita administrar de forma eficiente la información operativa y administrativa del Club Villa del Parque II.

3.2 Objetivos Específicos

- Gestionar el registro de socios.
 - Administrar las disciplinas deportivas.
 - Registrar inscripciones deportivas.
 - Controlar pagos de cuotas.
 - Registrar invitados.
 - Generar reportes administrativos.
 - Implementar control de acceso mediante usuarios.
 - Facilitar la consulta de información.
 - Centralizar la información en una base de datos relacional.
-

4. ALCANCE DEL SISTEMA

El sistema contempla las siguientes funcionalidades:

Gestión de Usuarios

Permite controlar el acceso mediante usuario y contraseña.

Gestión de Socios

Permite:

- Alta de socios.
- Modificación de socios.
- Baja lógica.
- Consulta de información.

Gestión de Deportes

Permite:

- Alta de deportes.
- Modificación de deportes.
- Baja lógica.
- Consulta de disciplinas.

Gestión de Inscripciones

Permite asociar socios con actividades deportivas.

Gestión de Pagos

Permite registrar:

- Cuotas sociales.
- Cuotas deportivas.
- Importes abonados.

Gestión de Invitados

Permite registrar personas no asociadas al club.

Gestión de Reportes

Permite generar:

- Estadísticas.
- Rendiciones mensuales.
- Consultas administrativas.

5. TECNOLOGÍAS UTILIZADAS

Python

Python es un lenguaje de programación de alto nivel ampliamente utilizado en el desarrollo de aplicaciones empresariales.

Ventajas:

- Sintaxis sencilla.
 - Gran cantidad de bibliotecas.
 - Facilidad de mantenimiento.
 - Amplia comunidad de desarrollo.
-

Tkinter

Tkinter es la biblioteca gráfica estándar de Python.

Fue utilizada para desarrollar:

- Formularios.
 - Ventanas.
 - Botones.
 - Tablas.
 - Cuadros de diálogo.
-

SQL Server

SQL Server fue utilizado como gestor de base de datos.

Características:

- Integridad referencial.
 - Seguridad.
 - Escalabilidad.
 - Consultas optimizadas.
-

PyODBC

PyODBC permite la conexión entre Python y SQL Server.

Su función principal es ejecutar consultas SQL desde la aplicación.

6. ARQUITECTURA DEL SISTEMA

El proyecto fue desarrollado utilizando una arquitectura por capas.

Capa de Presentación

Ubicación:

vistas/

Responsabilidades:

- Mostrar información.
 - Capturar datos.
 - Interactuar con el usuario.
-

Capa de Negocio

Ubicación:

modelos/

Responsabilidades:

- Procesamiento de información.
 - Reglas de negocio.
 - Consultas SQL.
-

Capa de Datos

Ubicación:

SQL Server

Responsabilidades:

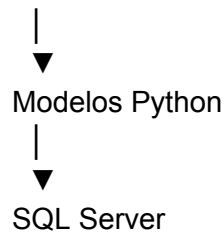
- Almacenamiento permanente.
 - Relaciones entre entidades.
 - Seguridad de los datos.
-

Flujo General

Usuario



Interfaz Tkinter



7. ANÁLISIS DE REQUERIMIENTOS

Requerimientos Funcionales

- 1: El sistema debe permitir iniciar sesión.
 - 2: El sistema debe permitir registrar socios.
 - 3: El sistema debe permitir modificar socios.
 - 4: El sistema debe permitir realizar bajas lógicas.
 - 5: El sistema debe permitir registrar deportes.
 - 6: El sistema debe permitir registrar inscripciones.
 - 7: El sistema debe permitir registrar pagos.
 - 8: El sistema debe permitir registrar invitados.
 - 9: El sistema debe generar reportes.
 - 10: El sistema debe mostrar estadísticas administrativas.
-

Requerimientos No Funcionales

- 1: La interfaz debe ser sencilla de utilizar.
- 2: La información debe almacenarse en SQL Server.
- 3: El sistema debe mantener integridad de datos.
- 4: El acceso debe estar protegido mediante credenciales.
- 5: La aplicación debe ejecutarse en sistemas Windows.

8. DISEÑO DE BASE DE DATOS

8.1 Introducción

La base de datos constituye el núcleo del sistema, ya que almacena toda la información relacionada con socios, deportes, inscripciones, pagos, invitados y usuarios.

Se utilizó un modelo relacional implementado en Microsoft SQL Server, garantizando:

- Integridad de los datos.
- Consistencia de la información.
- Facilidad de consulta.
- Escalabilidad futura.
- Seguridad en el almacenamiento.

La base de datos desarrollada recibe el nombre:

ClubVillaDelParqueII

8.2 Modelo Entidad Relación

El modelo entidad relación (MER) fue diseñado para representar los elementos fundamentales del negocio y sus relaciones.

Las principales entidades identificadas son:

- Usuarios
- Socios
- Deportes
- Inscripciones
- Pagos
- Invitados

Estas entidades permiten representar todas las operaciones requeridas por el club.

8.3 Entidad Usuarios

Descripción

Almacena las credenciales necesarias para acceder al sistema.

Permite implementar control de acceso restringido, cumpliendo uno de los requisitos principales del proyecto.

Estructura

Campo	Tipo	Descripción
IdUsuario	INT	Identificador único
Usuario	VARCHAR(50)	Nombre de usuario
Password	VARCHAR(100)	Contraseña
Rol	VARCHAR(50)	Tipo de usuario

Justificación

Esta tabla permite:

- Controlar accesos.
 - Identificar operadores.
 - Limitar funciones según rol.
 - Registrar usuarios autorizados.
-

8.4 Entidad Socios

Descripción

Representa a todas las personas asociadas al club.

Es una de las entidades centrales del sistema.

Estructura

Campo	Tipo
IdSocio	INT

Nombre	VARCHAR(50)
Apellido	VARCHAR(50)
DNI	VARCHAR(20)
Telefono	VARCHAR(30)
Direccion	VARCHAR(100)
Activo	BIT

Justificación de los Campos

IdSocio

Clave primaria utilizada para identificar de forma única a cada socio.

Nombre

Permite registrar el nombre del socio.

Apellido

Permite registrar el apellido del socio.

DNI

Documento Nacional de Identidad.

Permite evitar registros duplicados.

Teléfono

Información de contacto.

Dirección

Domicilio registrado.

Activo

Permite implementar bajas lógicas.

Valores posibles:

1 = Activo

0 = Inactivo

Ventajas de la Baja Lógica

- Conservación del historial.
 - Integridad referencial.
 - Evita pérdida de información.
 - Facilita auditorías.
-

8.5 Entidad Deportes

Descripción

Contiene las disciplinas deportivas ofrecidas por el club.

Estructura

Campo	Tipo
IdDeporte	INT
Nombre	VARCHAR(50)
CuotaMensual	DECIMAL(10,2)
Activo	BIT

Justificación

Permite administrar dinámicamente los deportes disponibles.

Ejemplos:

- Fútbol
 - Básquet
 - Tenis
-

Cuota Mensual

Representa el valor económico de la actividad deportiva.

Se utiliza para:

- Consultas administrativas.
 - Gestión de pagos.
 - Reportes financieros.
-

8.6 Entidad Inscripciones

Descripción

Representa la relación entre socios y deportes.

Un socio puede practicar múltiples deportes.

Un deporte puede tener múltiples socios.

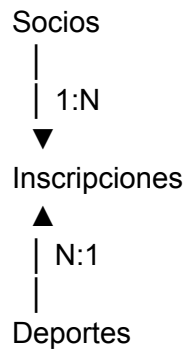
Por esta razón se implementó una tabla intermedia.

Estructura

Campo	Tipo
IdInscripcion	INT
IdSocio	INT

IdDeporte	INT
FechaInscripcion	DATETIME
Activo	BIT

Relación



Justificación

Permite:

- Registrar múltiples actividades por socio.
 - Conocer participantes por deporte.
 - Obtener estadísticas.
-

8.7 Entidad Pagos

Descripción

Almacena todos los pagos realizados por socios.

Estructura

Campo	Tipo
IdPago	INT
IdSocio	INT

Concepto	VARCHAR(50)
Importe	DECIMAL(10,2)
FechaPag	DATETIME
o	

Conceptos Posibles

- Cuota Social
 - Cuota Deportiva
 - Actividad Especial
 - Evento Deportivo
-

Importancia

Esta tabla es fundamental para:

- Rendiciones mensuales.
 - Estadísticas financieras.
 - Comprobantes de pago.
-

8.8 Entidad Invitados

Descripción

Representa a personas que utilizan instalaciones deportivas sin ser socios.

Estructura

Campo	Tipo
IdInvitado	INT
Nombre	VARCHAR(50)
Apellido	VARCHAR(50)

DNI	VARCHAR(20)
IdDeporte	INT
Importe	DECIMAL(10,2)
FechaVisita	DATETIME

Justificación

Permite:

- Llevar control de visitantes.
 - Registrar ingresos extraordinarios.
 - Obtener estadísticas de utilización.
-

8.9 Relaciones del Modelo

Usuarios

No posee dependencias directas.

Socios

Relacionada con:

- Inscripciones
 - Pagos
-

Deportes

Relacionada con:

- Inscripciones
 - Invitados
-

Inscripciones

Relaciona:

- Socios
 - Deportes
-

Pagos

Relacionada con:

- Socios
-

Invitados

Relacionada con:

- Deportes
-

8.10 Integridad Referencial

Se implementaron claves foráneas para garantizar la consistencia de los datos.

Ejemplos:

FOREIGN KEY (IdSocio)
REFERENCES Socios(IdSocio)

FOREIGN KEY (IdDeporte)
REFERENCES Deportes(IdDeporte)

Beneficios

- Evita registros huérfanos.
 - Garantiza consistencia.
 - Mejora la calidad de los datos.
-

8.11 Normalización

La base de datos fue diseñada respetando las primeras formas normales.

Primera Forma Normal (1FN)

Cada campo contiene valores atómicos.

Ejemplo:

Correcto:

Teléfono: 11-1234-5678

Incorrecto:

Teléfono: 11-1234-5678 / 11-9999-8888

Segunda Forma Normal (2FN)

Todos los atributos dependen completamente de la clave primaria.

Tercera Forma Normal (3FN)

No existen dependencias transitivas entre atributos.

Esto reduce:

- Redundancia.
 - Inconsistencias.
 - Problemas de actualización.
-

8.12 Ventajas del Diseño Propuesto

El modelo implementado permite:

- Escalabilidad.
- Mantenimiento sencillo.
- Facilidad de consultas.
- Integridad de información.

- Generación eficiente de reportes.
 - Posibilidad de futuras ampliaciones.
-

8.13 Posibles Ampliaciones Futuras

El diseño permite incorporar fácilmente nuevas entidades:

- Profesores
- Empleados
- Canchas
- Reservas
- Torneos
- Asistencia
- Carnet digital
- Facturación electrónica

Sin necesidad de rediseñar completamente la base de datos.

8.14 Conclusión del Diseño de Base de Datos

La estructura desarrollada cumple con todos los requerimientos planteados para la gestión administrativa del Club Villa del Parque II.

El modelo relacional implementado garantiza:

- Consistencia.
- Seguridad.
- Integridad.
- Escalabilidad.

Constituyendo una base sólida para el funcionamiento del sistema.

9. DESCRIPCIÓN DE LOS MÓDULOS DEL SISTEMA

9.1 Introducción

El Sistema de Gestión Deportiva Club Villa del Parque II fue desarrollado mediante módulos independientes, cada uno especializado en una tarea específica de la administración del club.

Esta división permite:

- Mejor organización del código.
- Mayor facilidad de mantenimiento.
- Escalabilidad futura.
- Mejor experiencia de usuario.

Cada módulo cumple una función determinada dentro del flujo general del sistema.

9.2 Módulo de Inicio de Sesión (Login)

Objetivo

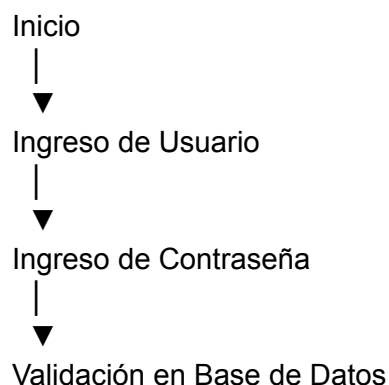
Controlar el acceso al sistema mediante autenticación de usuarios.

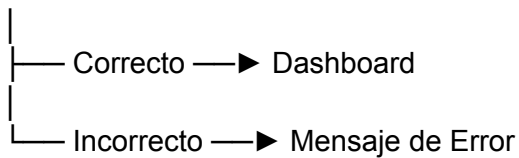
Este módulo constituye la primera barrera de seguridad de la aplicación.

Funcionalidades

- Solicitud de usuario.
 - Solicitud de contraseña.
 - Validación de credenciales.
 - Apertura del sistema únicamente si la autenticación es correcta.
-

Flujo de Operación





Ventajas

- Evita accesos no autorizados.
 - Protege la información almacenada.
 - Cumple con el requisito de acceso restringido.
-

9.3 Módulo Dashboard

Objetivo

Actuar como centro principal de navegación del sistema.

Una vez autenticado, el usuario accede a esta pantalla.

Funcionalidades

Permite acceder a:

- Gestión de Socios
 - Gestión de Deportes
 - Gestión de Inscripciones
 - Gestión de Pagos
 - Gestión de Invitados
 - Reportes
 - Salida del Sistema
-

Beneficios

- Acceso rápido a todos los módulos.
- Interfaz simple.
- Navegación intuitiva.

Flujo

Login



Dashboard



- Socios
- Deportes
- Inscripciones
- Pagos
- Invitados
- Reportes

9.4 Módulo Gestión de Socios

Objetivo

Administrar toda la información relacionada con los socios del club.

Funcionalidades Implementadas

Alta de Socios

Permite registrar nuevos socios.

Datos requeridos:

- Nombre
 - Apellido
 - DNI
 - Teléfono
 - Dirección
-

Modificación

Permite actualizar información existente.

Casos frecuentes:

- Cambio de domicilio.
 - Cambio de teléfono.
 - Corrección de datos.
-

Baja Lógica

Permite desactivar socios sin eliminarlos físicamente.

La información permanece almacenada para futuras consultas.

Consulta

Permite visualizar todos los socios registrados.

Beneficios

- Centralización de información.
 - Fácil acceso a los datos.
 - Historial preservado.
-

9.5 Módulo Gestión de Deportes

Objetivo

Administrar las disciplinas deportivas ofrecidas por el club.

Funcionalidades

Alta

Registro de nuevas actividades deportivas.

Ejemplos:

- Fútbol
 - Básquet
 - Tenis
-

Modificación

Actualización de:

- Nombre del deporte
 - Cuota mensual
-

Baja Lógica

Desactivación de actividades que ya no se ofrecen.

Consulta

Visualización de deportes registrados.

Beneficios

- Flexibilidad administrativa.
 - Gestión dinámica de actividades.
-

9.6 Módulo Inscripciones

Objetivo

Relacionar socios con actividades deportivas.

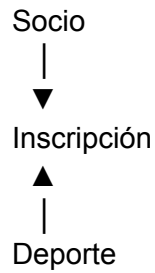
Funcionalidades

Registro de Inscripciones

Permite:

- Seleccionar un socio.
 - Seleccionar un deporte.
 - Registrar la inscripción.
-

Relación Implementada



Beneficios

- Control de participación.
 - Obtención de estadísticas.
 - Seguimiento de actividades.
-

Ejemplo

Socio:

Juan Pérez

Inscrito en:

Fútbol

Tenis

9.7 Módulo Pagos

Objetivo

Registrar los pagos realizados por los socios.

Funcionalidades

Registro de Pago

Permite almacenar:

- Socio
- Concepto
- Importe
- Fecha

Conceptos Disponibles

- Cuota Social
- Cuota Deportiva

Comprobante

Luego de registrar un pago se muestra una constancia en pantalla.

La misma contiene:

- Nombre del socio
- Concepto
- Importe
- Fecha

Beneficios

- Control financiero.
- Historial de pagos.
- Rendición económica.

9.8 Módulo Invitados

Objetivo

Registrar visitantes que utilizan las instalaciones deportivas sin ser socios.

Funcionalidades

Registro

Datos almacenados:

- Nombre
 - Apellido
 - DNI
 - Deporte
 - Importe abonado
 - Fecha
-

Beneficios

- Control de visitantes.
 - Registro de ingresos extraordinarios.
 - Estadísticas de utilización.
-

Ejemplo

Invitado:
Carlos López

Actividad:
Tenis

Importe:
\$5.000

9.9 Módulo Reportes

Objetivo

Generar información administrativa y financiera para la toma de decisiones.

Funcionalidades

Cantidad de Socios Activos

Permite conocer la cantidad actual de socios habilitados.

Cantidad de Inscripciones

Indica cuántas actividades deportivas se encuentran registradas.

Cantidad de Invitados

Muestra el número total de visitantes registrados.

Pagos del Mes

Calcula la cantidad de pagos realizados durante el mes actual.

Recaudación Total

Muestra:

- Recaudación por socios.
 - Recaudación por invitados.
-

Recaudación por Concepto

Permite identificar los ingresos según:

- Cuota Social
 - Cuota Deportiva
-

Deporte Más Popular

Determina la actividad con mayor cantidad de inscriptos.

Socios por Deporte

Muestra cuántos socios practican cada disciplina.

Invitados por Deporte

Permite conocer qué actividades reciben más visitantes.

9.10 Ventajas del Sistema Modular

La división por módulos permite:

- Menor complejidad.
 - Mayor mantenimiento.
 - Escalabilidad.
 - Reutilización de código.
-

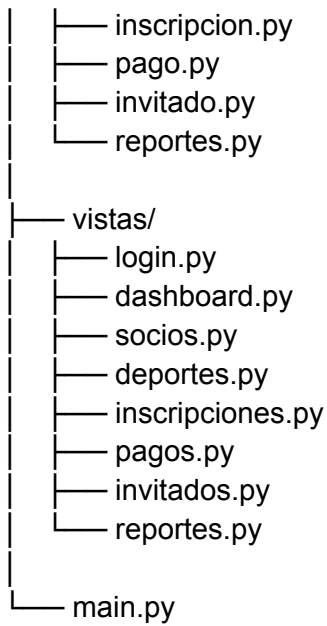
10. IMPLEMENTACIÓN DEL SISTEMA

10.1 Estructura de Carpetas

El proyecto fue organizado utilizando una estructura clara y ordenada.

club-villa-del-parque-ii/

```
|
|— config/
|   — conexion.py
|
|— modelos/
|   — usuario.py
|   — socio.py
|   — deporte.py
```



10.2 Archivo Principal

main.py

Es el punto de entrada del sistema.

Responsabilidades:

- Iniciar la aplicación.
- Mostrar la ventana de login.
- Comenzar el flujo del sistema.

10.3 Configuración de Base de Datos

conexion.py

Responsable de:

- Establecer conexión con SQL Server.
- Centralizar la configuración.
- Permitir reutilización en todos los módulos.

Servidor utilizado:

localhost\SQLEXPRESS

Base de datos:

10.4 Patrón Utilizado

Se implementó una separación entre:

Vistas

Interfaz gráfica.

Modelos

Acceso a datos.

Esto permite mantener una arquitectura organizada y fácilmente mantenible.

10.5 Manejo de Eventos

La aplicación utiliza eventos de Tkinter.

Ejemplos:

- Click en botones.
 - Selección de filas.
 - Confirmaciones.
 - Mensajes emergentes.
-

10.6 Validaciones

Durante el desarrollo se incorporaron validaciones para evitar errores comunes:

- Selección obligatoria antes de modificar.
 - Selección obligatoria antes de eliminar.
 - Confirmaciones de borrado.
 - Control de acceso mediante login.
-

10.7 Baja Lógica

Tanto socios como deportes utilizan bajas lógicas.

Esto implica:

Activo = 1 → Registro habilitado

Activo = 0 → Registro deshabilitado

Beneficios:

- Conservación histórica.
- Integridad referencial.
- Auditoría futura.

11. CONSULTAS SQL IMPLEMENTADAS

Introducción

Uno de los requisitos del proyecto fue la utilización de consultas SQL para la obtención, manipulación y análisis de la información almacenada.

Las consultas implementadas permiten realizar operaciones de:

- Consulta de datos.
- Inserción de registros.
- Actualización de información.
- Bajas lógicas.
- Obtención de estadísticas.
- Generación de reportes administrativos.

A continuación se describen las principales consultas utilizadas en el sistema.

Consulta 1 – Listado de Socios Activos

Objetivo

Mostrar todos los socios habilitados.

```
SELECT *  
FROM Socios  
WHERE Activo = 1;
```

Utilidad

Permite visualizar únicamente los socios vigentes.

Consulta 2 – Cantidad de Socios Activos

Objetivo

Conocer el número total de socios habilitados.

```
SELECT COUNT(*)  
FROM Socios  
WHERE Activo = 1;
```

Utilidad

Utilizada en el módulo de reportes.

Consulta 3 – Buscar Socio por DNI

Objetivo

Localizar rápidamente un socio.

```
SELECT *  
FROM Socios  
WHERE DNI LIKE '%12345678%';
```

Utilidad

Facilita búsquedas administrativas.

Consulta 4 – Listado Completo de Deportes

Objetivo

Mostrar todas las disciplinas registradas.

```
SELECT *  
FROM Deportes;
```

Consulta 5 – Deportes Activos

Objetivo

Mostrar únicamente deportes disponibles.

```
SELECT *  
FROM Deportes  
WHERE Activo = 1;
```

Consulta 6 – Cantidad de Deportes Registrados

Objetivo

Determinar cuántas disciplinas existen.

```
SELECT COUNT(*)  
FROM Deportes;
```

Consulta 7 – Cantidad de Inscripciones

Objetivo

Contabilizar todas las inscripciones deportivas.

```
SELECT COUNT(*)  
FROM Inscripciones;
```

Consulta 8 – Socios Inscriptos por Deporte

Objetivo

Conocer cuántos socios participan en cada disciplina.

```
SELECT
  D.Nombre,
  COUNT(*) AS Cantidad
FROM Inscripciones I
INNER JOIN Deportes D
ON I.IdDeporte = D.IdDeporte
GROUP BY D.Nombre;
```

Consulta 9 – Deporte Más Popular

Objetivo

Determinar la actividad con mayor participación.

```
SELECT TOP 1
  D.Nombre,
  COUNT(*) AS Cantidad
FROM Inscripciones I
INNER JOIN Deportes D
ON I.IdDeporte = D.IdDeporte
GROUP BY D.Nombre
ORDER BY COUNT(*) DESC;
```

Consulta 10 – Total Recaudado

Objetivo

Calcular ingresos generales.

```
SELECT SUM(Importe)
FROM Pagos;
```

Consulta 11 – Pagos del Mes Actual

Objetivo

Conocer actividad financiera mensual.

```
SELECT COUNT(*)  
FROM Pagos  
WHERE MONTH(FechaPago) = MONTH(GETDATE())  
AND YEAR(FechaPago) = YEAR(GETDATE());
```

Consulta 12 – Recaudación por Concepto

Objetivo

Separar ingresos según tipo.

```
SELECT  
    Concepto,  
    SUM(Importe) AS Total  
FROM Pagos  
GROUP BY Concepto;
```

Consulta 13 – Total Recaudado por Cuota Social

Objetivo

Analizar ingresos por membresías.

```
SELECT SUM(Importe)  
FROM Pagos  
WHERE Concepto = 'Cuota Social';
```

Consulta 14 – Total Recaudado por Cuota Deportiva

Objetivo

Analizar ingresos por actividades.

```
SELECT SUM(Importe)
FROM Pagos
WHERE Concepto = 'Cuota Deportiva';
```

Consulta 15 – Cantidad de Invitados

Objetivo

Conocer visitantes registrados.

```
SELECT COUNT(*)
FROM Invitados;
```

Consulta 16 – Total Recaudado por Invitados

Objetivo

Calcular ingresos extraordinarios.

```
SELECT SUM(Importe)
FROM Invitados;
```

Consulta 17 – Invitados por Deporte

Objetivo

Determinar qué actividades reciben más visitantes.

```
SELECT
    D.Nombre,
    COUNT(*) AS Cantidad
FROM Invitados I
INNER JOIN Deportes D
ON I.IdDeporte = D.IdDeporte
GROUP BY D.Nombre;
```

Consulta 18 – Socios con Pagos Registrados

Objetivo

Obtener listado de socios que realizaron pagos.

```
SELECT DISTINCT
    S.Nombre,
    S.Apellido
FROM Socios S
INNER JOIN Pagos P
ON S.IdSocio = P.IdSocio;
```

Consulta 19 – Historial de Pagos de un Socio

Objetivo

Consultar pagos individuales.

```
SELECT *
FROM Pagos
WHERE IdSocio = 1;
```

Consulta 20 – Listado Completo de Inscripciones

Objetivo

Visualizar todas las relaciones socio-deporte.

```
SELECT
  S.Nombre,
  S.Apellido,
  D.Nombre AS Deporte
FROM Inscripciones I
INNER JOIN Socios S
ON I.IdSocio = S.IdSocio
INNER JOIN Deportes D
ON I.IdDeporte = D.IdDeporte;
```

12. SEGURIDAD Y CONTROL DE ACCESO

12.1 Introducción

La seguridad es un aspecto fundamental en cualquier sistema de información.

Para este proyecto se implementó un mecanismo básico de autenticación que restringe el acceso únicamente a usuarios autorizados.

12.2 Autenticación

El sistema solicita:

- Usuario
- Contraseña

Antes de permitir el acceso.

La validación se realiza consultando la tabla Usuarios de la base de datos.

12.3 Beneficios del Control de Acceso

- Protección de información.
 - Restricción de acceso.
 - Identificación de operadores.
 - Mayor seguridad administrativa.
-

12.4 Flujo de Validación

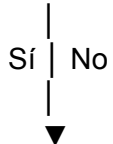
Usuario ingresa credenciales



Consulta SQL



¿Existe usuario?



Dashboard

13. PRUEBAS REALIZADAS

Introducción

Durante el desarrollo se realizaron diversas pruebas para verificar el correcto funcionamiento de cada módulo.

Prueba 1 – Inicio de Sesión

Objetivo

Verificar autenticación.

Procedimiento

Ingresar usuario válido.

Resultado Esperado

Acceso al sistema.

Resultado Obtenido

Correcto.

Prueba 2 – Login Incorrecto

Objetivo

Verificar rechazo de accesos inválidos.

Resultado Esperado

Mensaje de error.

Resultado Obtenido

Correcto.

Prueba 3 – Alta de Socio

Objetivo

Registrar un nuevo socio.

Resultado Esperado

Inserción exitosa.

Resultado Obtenido

Correcto.

Prueba 4 – Modificación de Socio

Resultado Esperado

Actualización de datos.

Resultado Obtenido

Correcto.

Prueba 5 – Baja Lógica de Socio

Resultado Esperado

Cambio de Activo = 1 a Activo = 0.

Resultado Obtenido

Correcto.

Prueba 6 – Alta de Deporte

Resultado Esperado

Registro exitoso.

Resultado Obtenido

Correcto.

Prueba 7 – Modificación de Deporte

Resultado Esperado

Actualización correcta.

Resultado Obtenido

Correcto.

Prueba 8 – Baja Lógica de Deporte

Resultado Esperado

Desactivación del deporte.

Resultado Obtenido

Correcto.

Prueba 9 – Registro de Inscripción**Resultado Esperado**

Asociación socio-deporte.

Resultado Obtenido

Correcto.

Prueba 10 – Registro de Pago**Resultado Esperado**

Almacenamiento del pago.

Resultado Obtenido

Correcto.

Prueba 11 – Registro de Invitado**Resultado Esperado**

Alta exitosa.

Resultado Obtenido

Correcto.

Prueba 12 – Generación de Reportes

Resultado Esperado

Visualización correcta de estadísticas.

Resultado Obtenido

Correcto.

Prueba 13 – Integridad de Base de Datos

Objetivo

Verificar relaciones entre tablas.

Resultado Obtenido

Correcto.

14. RESULTADOS OBTENIDOS

La implementación del sistema permitió alcanzar los objetivos planteados al inicio del proyecto.

Se logró desarrollar una aplicación capaz de:

- Gestionar socios.
- Gestionar deportes.
- Administrar inscripciones.
- Registrar pagos.
- Controlar invitados.
- Generar reportes.
- Proteger el acceso mediante autenticación.

Además, se consiguió integrar Python con SQL Server mediante PyODBC, aplicando conocimientos de programación y bases de datos relacionales.

15. CONCLUSIONES

El desarrollo del Sistema de Gestión Deportiva Club Villa del Parque II permitió aplicar de manera práctica conocimientos adquiridos durante la cursada, integrando programación, diseño de bases de datos y desarrollo de interfaces gráficas.

Entre los principales aprendizajes obtenidos se destacan:

- Modelado de bases de datos relacionales.
- Diseño de interfaces de usuario.
- Programación orientada a objetos.
- Consultas SQL.
- Integración entre Python y SQL Server.
- Gestión de proyectos de software.

El sistema desarrollado cumple con los requerimientos planteados inicialmente y constituye una solución funcional para la administración de un club deportivo.

16. MEJORAS FUTURAS

16.1 Introducción

Si bien el sistema desarrollado cumple con los requerimientos planteados para la administración del Club Villa del Parque II, durante el análisis y desarrollo se identificaron diversas funcionalidades que podrían incorporarse en futuras versiones para ampliar sus capacidades.

La estructura modular implementada facilita la incorporación de nuevas características sin necesidad de rediseñar completamente la aplicación.

16.2 Exportación de Reportes a PDF

Actualmente los reportes se visualizan dentro del sistema.

Como mejora futura podría implementarse la generación automática de archivos PDF con:

- Rendiciones mensuales.
- Listado de socios.
- Listado de deportes.
- Estadísticas generales.
- Historial de pagos.

Beneficios:

- Facilidad de impresión.
 - Compartición de información.
 - Archivado digital.
-

16.3 Exportación a Excel

Una mejora muy utilizada en entornos administrativos consiste en exportar datos a hojas de cálculo.

Ejemplos:

- Socios activos.
- Pagos del mes.
- Inscripciones.
- Invitados.

Beneficios:

- Análisis externo.
 - Integración con herramientas contables.
 - Mayor flexibilidad para los administradores.
-

16.4 Gestión de Profesores

Actualmente el sistema administra socios y deportes.

Una ampliación natural sería incorporar:

Tabla Profesores

Campos sugeridos:

- IdProfesor
- Nombre
- Apellido
- DNI
- Teléfono
- Especialidad

Esto permitiría asociar profesores a actividades deportivas.

16.5 Gestión de Empleados

Permitiría administrar:

- Recepcionistas.
- Administrativos.
- Personal de mantenimiento.

Información posible:

- Horarios.
 - Cargos.
 - Sueldos.
 - Datos personales.
-

16.6 Control de Asistencia

Una funcionalidad muy útil para clubes deportivos consiste en registrar la asistencia de los socios.

Beneficios:

- Seguimiento de participación.
 - Estadísticas de concurrencia.
 - Control de actividades.
-

16.7 Carnet Digital

Actualmente los socios se identifican mediante su registro en la base de datos.

Una versión futura podría generar:

- Carnet digital.
- Código QR.
- Identificación rápida.

Beneficios:

- Modernización.
 - Agilidad de acceso.
 - Menor uso de papel.
-

16.8 Reserva de Canchas

Podría incorporarse un sistema para reservar:

- Canchas de fútbol.
- Canchas de tenis.
- Gimnasio.

Información necesaria:

- Fecha.
 - Horario.
 - Usuario responsable.
-

16.9 Torneos Deportivos

Una mejora interesante sería la gestión de competencias.

Posibilidades:

- Creación de torneos.
 - Equipos participantes.
 - Fixture.
 - Resultados.
 - Tabla de posiciones.
-

16.10 Dashboard Gráfico

Actualmente los reportes se presentan mediante texto.

Podrían incorporarse gráficos utilizando bibliotecas como:

- Matplotlib
- Plotly

Ejemplos:

- Socios por deporte.
 - Recaudación mensual.
 - Evolución de pagos.
-

16.11 Versión Web

Actualmente el sistema funciona como aplicación de escritorio.

Una futura migración podría realizarse mediante:

- Flask
- Django

Beneficios:

- Acceso remoto.
 - Multiusuario.
 - Uso desde cualquier dispositivo.
-

16.12 Aplicación Móvil

Otra mejora futura sería una aplicación para Android o iOS.

Funciones posibles:

- Consulta de cuotas.
 - Inscripción a actividades.
 - Consulta de horarios.
 - Visualización de comprobantes.
-

17. GLOSARIO DE TÉRMINOS

Base de Datos

Conjunto organizado de información almacenada electrónicamente.

SQL

Lenguaje utilizado para consultar y administrar bases de datos.

SQL Server

Sistema gestor de bases de datos desarrollado por Microsoft.

PyODBC

Biblioteca de Python que permite conectarse a SQL Server.

CRUD

Acrónimo de:

- Create
- Read
- Update
- Delete

Representa las operaciones básicas sobre datos.

Clave Primaria

Campo que identifica de forma única cada registro.

Ejemplo:

IdSocio

Clave Foránea

Campo utilizado para relacionar tablas.

Ejemplo:

IdDeporte

en la tabla Inscripciones.

Integridad Referencial

Mecanismo que garantiza que las relaciones entre tablas sean válidas.

Baja Lógica

Proceso mediante el cual un registro se desactiva sin eliminarse físicamente.

Ejemplo:

Activo = 0

Interfaz Gráfica

Conjunto de ventanas y elementos visuales utilizados por el usuario.

Dashboard

Pantalla principal desde la cual se accede a los distintos módulos.

Reporte

Información generada por el sistema con fines administrativos o estadísticos.

18. DICCIONARIO DE DATOS

Tabla Usuarios

Campo	Tipo	Descripción
IdUsuario	INT	Identificador único
Usuario	VARCHAR(50)	Nombre de usuario
Password	VARCHAR(100)	Contraseña
Rol	VARCHAR(50)	Perfil de acceso

Tabla Socios

Campo	Tipo	Descripción
IdSocio	INT	Identificador único
Nombre	VARCHAR(50)	Nombre del socio
Apellido	VARCHAR(50)	Apellido
DNI	VARCHAR(20)	Documento
Telefono	VARCHAR(30)	Contacto
Direccion	VARCHAR(100)	Domicilio
Activo	BIT	Estado del socio

Tabla Deportes

Campo	Tipo	Descripción
IdDeporte	INT	Identificador único
Nombre	VARCHAR(50)	Nombre del deporte
CuotaMensual	DECIMAL(10,2)	Cuota mensual
Activo	BIT	Estado del deporte

Tabla Inscripciones

Campo	Tipo	Descripción
IdInscripcion	INT	Identificador
IdSocio	INT	Socio asociado
IdDeporte	INT	Deporte asociado
FechaInscripcion	DATETIME	Fecha de alta
Activo	BIT	Estado

Tabla Pagos

Campo	Tipo	Descripción
IdPago	INT	Identificador
IdSocio	INT	Socio que realizó el pago
Concepto	VARCHAR(50)	Motivo del pago
Importe	DECIMAL(10,2)	Valor abonado
FechaPago	DATETIME	Fecha de pago

Tabla Invitados

Campo	Tipo	Descripción
IdInvitado	INT	Identificador
Nombre	VARCHAR(50)	Nombre
Apellido	VARCHAR(50)	Apellido
DNI	VARCHAR(20)	Documento
IdDeporte	INT	Actividad realizada
Importe	DECIMAL(10,2)	Monto abonado
FechaVisita	DATETIME	Fecha de ingreso

19. ANEXOS

Anexo A – Script de Creación de Base de Datos

En este apartado se incluye el script SQL completo utilizado para generar la estructura de la base de datos.

Elementos incluidos:

- **Creación de tablas.**

Crear Base de Datos

```
Crear Base de Datos  
CREATE DATABASE ClubVillaDelParqueII;  
GO
```

Tabla Usuarios

```
USE ClubVillaDelParqueII;  
GO  
Tabla Usuarios  
CREATE TABLE Usuarios  
(  
    IdUsuario INT IDENTITY(1,1) PRIMARY KEY,  
    Usuario VARCHAR(50) NOT NULL UNIQUE,  
    Password VARCHAR(100) NOT NULL,  
    Rol VARCHAR(50) NOT NULL  
);
```

Tabla Socios

```
CREATE TABLE Socios
(
    IdSocio INT IDENTITY(1,1) PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL,
    Apellido VARCHAR(50) NOT NULL,
    DNI VARCHAR(20) NOT NULL UNIQUE,
    Telefono VARCHAR(30),
    Direccion VARCHAR(100),
    Activo BIT NOT NULL DEFAULT 1
);
```

Tabla Deportes

```
CREATE TABLE Deportes
(
    IdDeporte INT IDENTITY(1,1) PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL,
    CuotaMensual DECIMAL(10,2) NOT NULL,
    Activo BIT NOT NULL DEFAULT 1
);
```

Tabla Inscripciones

```
CREATE TABLE Inscripciones
(
    IdInscripcion INT IDENTITY(1,1) PRIMARY KEY,

    IdSocio INT NOT NULL,

    IdDeporte INT NOT NULL,

    FechaInscripcion DATETIME DEFAULT GETDATE(),

    Activo BIT DEFAULT 1,

    CONSTRAINT FK_Inscripciones_Socios
        FOREIGN KEY (IdSocio)
        REFERENCES Socios(IdSocio),

    CONSTRAINT FK_Inscripciones_Deportes
        FOREIGN KEY (IdDeporte)
        REFERENCES Deportes(IdDeporte)
);
```

Tabla Pagos

```
CREATE TABLE Pagos
(
    IdPago INT IDENTITY(1,1) PRIMARY KEY,

    IdSocio INT NOT NULL,

    Concepto VARCHAR(50) NOT NULL,

    Importe DECIMAL(10,2) NOT NULL,

    FechaPago DATETIME DEFAULT GETDATE(),

    CONSTRAINT FK_Pagos_Socios
        FOREIGN KEY (IdSocio)
        REFERENCES Socios(IdSocio)
);
```

Tabla Invitados

```
CREATE TABLE Invitados
(
    IdInvitado INT IDENTITY(1,1) PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL,
    Apellido VARCHAR(50) NOT NULL,
    DNI VARCHAR(20),
    IdDeporte INT NOT NULL,
    Importe DECIMAL(10,2) NOT NULL,
    FechaVisita DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Invitados_Deportes
        FOREIGN KEY (IdDeporte)
        REFERENCES Deportes(IdDeporte)
);
```

- DATOS INICIALES

Usuario Administrador

```
INSERT INTO Usuarios
(
    Usuario,
    Password,
    Rol
)
VALUES
(
    'admin',
    'admin123',
    'Administrador'
);
```


Deportes Iniciales

```
INSERT INTO Deportes
(
    Nombre,
    CuotaMensual
)
VALUES
('Fútbol', 15000);

INSERT INTO Deportes
(
    Nombre,
    CuotaMensual
)
VALUES
('Básquet', 18000);

INSERT INTO Deportes
(
    Nombre,
    CuotaMensual
)
VALUES
('Tenis', 22000);
```

CONSULTAS DE PRUEBA

```
Ver usuarios
SELECT *
FROM Usuarios;
Ver socios
SELECT *
FROM Socios;
Ver deportes
SELECT *
FROM Deportes;
Ver inscripciones
SELECT *
FROM Inscripciones;
Ver pagos
SELECT *
FROM Pagos;
Ver invitados
SELECT *
FROM Invitados;
```

CONSULTAS UTILIZADAS POR EL SISTEMA

```
Login
SELECT
    Usuario,
    Rol
FROM Usuarios
WHERE Usuario = ?
AND Password = ?;
Alta de Socio
INSERT INTO Socios
(
    Nombre,
    Apellido,
    DNI,
    Telefono,
    Direccion
)
VALUES
(
    ?, ?, ?, ?, ?
);
```

Modificar Socio

```
UPDATE Socios
SET
    Nombre = ?,
    Apellido = ?,
    DNI = ?,
    Telefono = ?,
    Direccion = ?
WHERE IdSocio = ?;
Baja Lógica de Socio
UPDATE Socios
SET Activo = 0
WHERE IdSocio = ?;
Alta de Deporte
INSERT INTO Deportes
(
    Nombre,
    CuotaMensual
)
VALUES
(
    ?, ?
);
```

Modificar Deporte

```
UPDATE Deportes
SET
    Nombre = ?,
    CuotaMensual = ?
WHERE IdDeporte = ?;
Baja Lógica de Deporte
UPDATE Deportes
SET Activo = 0
WHERE IdDeporte = ?;
```

Registrar Inscripción

```
INSERT INTO Inscripciones
(
    IdSocio,
    IdDeporte
)
VALUES
(
    ?, ?
);
```

Registrar Inscripción

```
INSERT INTO Inscripciones
(
    IdSocio,
    IdDeporte
)
VALUES
(
    ?, ?
);
```

Registrar Pago

```
INSERT INTO Pagos
(
    IdSocio,
    Concepto,
    Importe
)
VALUES
(
    ?, ?, ?
);
```

Registrar Invitado

```
INSERT INTO Invitados
(
    Nombre,
    Apellido,
    DNI,
    IdDeporte,
    Importe
)
VALUES
(
    ?, ?, ?, ?, ?
);
```

Anexo B – Código Fuente

Se adjunta el código fuente desarrollado en Python.

Módulos principales:

- Login

```
vistas > login.py > VentanaLogin > Ingresar

1  import tkinter as tk
2  from tkinter import messagebox
3
4  from modelos.usuario import Usuario
5  from vistas.dashboard import VentanaDashboard
6
7  class VentanaLogin:
8
9      def __init__(self):
10
11          self.ventana = tk.Tk()
12
13          self.ventana.title(
14              "Club Villa del Parque II"
15          )
16
17          self.ventana.geometry("400x250")
18
19          self.crear_componentes()
20
21          self.ventana.mainloop()
22
23      def crear_componentes(self):
24
25          tk.Label(
26              self.ventana,
27              text="Club Villa del Parque II",
28              font=("Arial", 16, "bold")
29          ).pack(pady=10)
30
31          tk.Label(
32              self.ventana,
33              text="Usuario"
34          ).pack()
35
36          self.txt_usuario = tk.Entry(
37              self.ventana
38          )
39
40          self.txt_usuario.pack()
41
42          tk.Label(
43              self.ventana,
44              text="Contraseña"
45          ).pack()
46
47          self.txt_password = tk.Entry(
48              self.ventana,
49              show="*"
50          )
51
52          self.txt_password.pack()
53
54          tk.Button(
55              self.ventana,
56              text="Ingresar",
57              command=self.ingresar
58          ).pack(pady=20)
59
60      def ingresar(self):
61
62          usuario = self.txt_usuario.get()
63
64          password = self.txt_password.get()
65
66          resultado = Usuario.validar(
67              usuario,
68              password
69          )
70
71          if resultado:
72
73              self.ventana.destroy()
74              VentanaDashboard(
75                  resultado.Usuario,
76                  resultado.NombreRol
77              )
78
79          else:
80
81              messagebox.showerror(
82                  "Error",
83                  "Usuario o contraseña incorrectos"
84              )
85
```

- **Dashboard**

```
import tkinter as tk

from vistas.socios import VentanaSocios

from vistas.deportes import VentanaDeportes

from vistas.inscripciones import VentanaInscripciones

from vistas.pagos import VentanaPagos

from vistas.reportes import VentanaReportes

from vistas.invitados import VentanaInvitados


class VentanaDashboard:

    def abrir_socios(self):

        VentanaSocios()

    def abrir_deportes(self):

        VentanaDeportes()

    def abrir_inscripciones(self):

        VentanaInscripciones()

    def abrir_pagos(self):

        VentanaPagos()
```

```
def abrir_reportes(self):

    VentanaReportes()

def abrir_invitados(self):

    VentanaInvitados()


def __init__(self, usuario, rol):

    self.ventana = tk.Tk()

    self.ventana.title(

        "Dashboard - Club Villa del Parque II"

    )

    self.ventana.geometry("600x450")

    self.usuario = usuario

    self.rol = rol

    self.crear_componentes()

    self.ventana.mainloop()


def crear_componentes(self):
```

```
titulo = tk.Label(  
    self.ventana,  
    text="CLUB VILLA DEL PARQUE II",  
    font=("Arial", 18, "bold")  
)
```

```
titulo.pack(pady=10)
```

```
lbl_usuario = tk.Label(  
    self.ventana,  
    text=f"Usuario: {self.usuario}"  
)
```

```
lbl_usuario.pack()
```

```
lbl_rol = tk.Label(  
    self.ventana,  
    text=f"Rol: {self.rol}"  
)
```

```
lbl_rol.pack(pady=10)
```

```
tk.Button(  
    self.ventana,
```



```
        text="Gestión de Socios",

        width=30,

        command=self.abrir_socios

    ).pack(pady=5)


    tk.Button(

        self.ventana,

        text="Gestión de Deportes",

        width=30,

        command=self.abrir_deportes

    ).pack(pady=5)


    tk.Button(

        self.ventana,

        text="Inscripciones",

        width=30,

        command=self.abrir_inscripciones

    ).pack(pady=5)


    tk.Button(

        self.ventana,

        text="Pagos",

        width=30,

        command=self.abrir_pagos

    ).pack(pady=5)
```

```

tk.Button(

    self.ventana,

    text="Invitados",

    width=30,

    command=self.abrir_invitados

).pack(pady=5)

tk.Button(

    self.ventana,

    text="Reportes",

    width=30,

    command=self.abrir_reportes

).pack(pady=5)


tk.Button(

    self.ventana,

    text="Salir",

    width=30,

    command=self.ventana.destroy

).pack(pady=20)

```

- **Socios**

```

import tkinter as tk

from tkinter import ttk

from tkinter import messagebox

```

```
from modelos.socio import Socio

class VentanaSocios:

    def __init__(self):

        self.id_seleccionado = None

        self.ventana = tk.Toplevel()

        self.ventana.title(

            "Gestión de Socios"

        )

        self.ventana.geometry("900x500")

        self.crear_componentes()

        self.cargar_socios()

    def crear_componentes(self):

        frame_formulario = tk.Frame(

            self.ventana
```

```
)

frame_formulario.pack(

    pady=10

)

# Nombre

tk.Label(

    frame_formulario,

    text="Nombre"

).grid(

    row=0,

    column=0

)

self.txt_nombre = tk.Entry(

    frame_formulario

)

self.txt_nombre.grid(

    row=0,

    column=1

)
```

```
# Apellido

tk.Label(

    frame_formulario,

    text="Apellido"

).grid(

    row=0,

    column=2

)

self.txt_apellido = tk.Entry(

    frame_formulario

)

self.txt_apellido.grid(

    row=0,

    column=3

)

# DNI

tk.Label(

    frame_formulario,

    text="DNI"

).grid(
```

```
        row=1,

        column=0

    )

    self.txt_dni = tk.Entry(

        frame_formulario

    )

    self.txt_dni.grid(

        row=1,

        column=1

    )

    # Teléfono

    tk.Label(

        frame_formulario,

        text="Teléfono"

    ).grid(

        row=1,

        column=2

    )

    self.txt_telefono = tk.Entry(

        frame_formulario
```

```
)

self.txt_telefono.grid(

    row=1,

    column=3

)

# Dirección

tk.Label(

    frame_formulario,

    text="Dirección"

).grid(

    row=2,

    column=0

)

self.txt_direccion = tk.Entry(

    frame_formulario,

    width=40

)

self.txt_direccion.grid(

    row=2,

    column=1,
```

```
        colspan=3

    )

    # Botones

    tk.Button(

        frame_formulario,

        text="Guardar Socio",

        command=self.guardar

    ).grid(

        row=3,

        column=0,

        pady=10

    )

    tk.Button(

        frame_formulario,

        text="Modificar",

        command=self.modificar

    ).grid(

        row=3,

        column=1,

        pady=10

    )
```



```
tk.Button(

    frame_formulario,

    text="Eliminar",

    command=self.eliminar

).grid(

    row=3,

    column=2,

    pady=10

)

tk.Button(

    frame_formulario,

    text="Refrescar",

    command=self.cargar_socios

).grid(

    row=3,

    column=3,

    pady=10

)

# Tabla

self.tabla = ttk.Treeview(

    self.ventana,

    columns=(
```

```
        "ID",

        "Nombre",

        "Apellido",

        "DNI",

        "Telefono",

        "Direccion",

        "Activo"

    ),

    show="headings"

)

self.tabla.heading("ID", text="ID")

self.tabla.heading("Nombre", text="Nombre")

self.tabla.heading("Apellido", text="Apellido")

self.tabla.heading("DNI", text="DNI")

self.tabla.heading("Telefono", text="Teléfono")

self.tabla.heading("Direccion", text="Dirección")

self.tabla.heading("Activo", text="Activo")

self.tabla.column("ID", width=60)

self.tabla.column("Nombre", width=120)

self.tabla.column("Apellido", width=120)

self.tabla.column("DNI", width=100)

self.tabla.column("Telefono", width=120)

self.tabla.column("Direccion", width=200)
```

```
self.tabla.column("Activo", width=80)

self.tabla.pack(

    fill="both",

    expand=True,

    padx=10,

    pady=10

)

self.tabla.bind(

    "<<TreeviewSelect>>",

    self.seleccionar_socio

)

def cargar_socios(self):

    for fila in self.tabla.get_children():

        self.tabla.delete(fila)

    socios = Socio.obtener_todos()

    for socio in socios:

        self.tabla.insert(

            "",
```

```
tk.END,  
  
values=(  
  
    socio.IdSocio,  
  
    socio.Nombre,  
  
    socio.Apellido,  
  
    socio.DNI,  
  
    socio.Telefono,  
  
    socio.Direccion,  
  
    socio.Activo  
  
    )  
  
    )  
  
def guardar(self):  
  
    Socio.agregar(  
  
        self.txt_nombre.get(),  
  
        self.txt_apellido.get(),  
  
        self.txt_dni.get(),  
  
        self.txt_telefono.get(),  
  
        self.txt_direccion.get()  
  
    )  
  
    messagebox.showinfo(  
  
        "Correcto",  
  
        "Socio agregado"
```

```
)

self.limpiar()

self.cargar_socios()

def seleccionar_socio(self, event):

    seleccion = self.tabla.selection()

    if not seleccion:

        return

    valores = self.tabla.item(

        seleccion[0]

    )["values"]

    self.id_seleccionado = valores[0]

    self.txt_nombre.delete(0, tk.END)

    self.txt_nombre.insert(0, valores[1])

    self.txt_apellido.delete(0, tk.END)

    self.txt_apellido.insert(0, valores[2])
```

```
self.txt_dni.delete(0, tk.END)

self.txt_dni.insert(0, valores[3])


self.txt_telefono.delete(0, tk.END)

self.txt_telefono.insert(0, valores[4])


self.txt_direccion.delete(0, tk.END)

self.txt_direccion.insert(0, valores[5])


def modificar(self):

    if self.id_seleccionado is None:

        messagebox.showwarning(

            "Aviso",

            "Seleccione un socio"

        )

        return

    Socio.actualizar(

        self.id_seleccionado,

        self.txt_nombre.get(),

        self.txt_apellido.get(),

        self.txt_dni.get(),
```

```
        self.txt_telefono.get(),

        self.txt_direccion.get()

    )

    self.cargar_socios()

    self.limpiar()

    self.id_seleccionado = None

    messagebox.showinfo(

        "Correcto",

        "Socio actualizado"

    )

def eliminar(self):

    if self.id_seleccionado is None:

        messagebox.showwarning(

            "Aviso",

            "Seleccione un socio"

        )

    return
```

```
respuesta = messagebox.askyesno(

    "Confirmar",

    "¿Desea dar de baja este socio?"

)

if respuesta:

    print("ID seleccionado:", self.id_seleccionado)

    Socio.baja_logica(

        self.id_seleccionado

    )

    self.cargar_socios()

    self.limpiar()

    self.id_seleccionado = None

    messagebox.showinfo(

        "Correcto",

        "Socio dado de baja"

    )

def limpiar(self):
```



```
self.txt_nombre.delete(0, tk.END)

self.txt_apellido.delete(0, tk.END)

self.txt_dni.delete(0, tk.END)

self.txt_telefono.delete(0, tk.END)

self.txt_direccion.delete(0, tk.END)
```

```
from config.conexion import obtener_conexion
```

```
class Socio:
```

```
    @staticmethod
```

```
    def obtener_todos():
```

```
        conexion = obtener_conexion()
```

```
        cursor = conexion.cursor()
```

```
        cursor.execute("""
```

```
            SELECT
```

```
                IdSocio,
```

```
                Nombre,
```

```
                Apellido,
```

```
                DNI,
```

```
                Telefono,
```

```
        Direccion,

        Activo

    FROM Socios

    ORDER BY Apellido, Nombre

    """

socios = cursor.fetchall()

conexion.close()

return socios

@staticmethod
def agregar(nombre, apellido, dni, telefono, direccion):

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        INSERT INTO Socios

        (

            Nombre,

            Apellido,

            DNI,
```

```
        Telefono,

        Direccion

    )

VALUES

    (?, ?, ?, ?, ?)

""" ,

(

    nombre,

    apellido,

    dni,

    telefono,

    direccion

))

conexion.commit()

conexion.close()

@staticmethod

def actualizar(id_socio, nombre, apellido, dni, telefono, direccion):

    conexion = obtener_conexion()

    cursor = conexion.cursor()
```

```
cursor.execute("""

UPDATE Socios

SET

    Nombre = ?,

    Apellido = ?,

    DNI = ?,

    Telefono = ?,

    Direccion = ?

WHERE IdSocio = ?

""",

(

    nombre,

    apellido,

    dni,

    telefono,

    direccion,

    id_socio

))

conexion.commit()

conexion.close()
```

```
@staticmethod
```

```
def baja_logica(id_socio):

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        UPDATE Socios

        SET Activo = 0

        WHERE IdSocio = ?

    """, (id_socio,))

    print("Filas afectadas:", cursor.rowcount)

    conexion.commit()

    conexion.close()

    @staticmethod

    def buscar_por_dni(dni):

        conexion = obtener_conexion()

        cursor = conexion.cursor()
```

```
cursor.execute("""

SELECT

    IdSocio,

    Nombre,

    Apellido,

    DNI,

    Telefono,

    Direccion,

    Activo

FROM Socios

WHERE DNI LIKE ?

""",

(f"%{dni}%",))

resultado = cursor.fetchall()

conexion.close()

return resultado

@staticmethod

def obtener_activos():

    conexion = obtener_conexion()

    cursor = conexion.cursor()
```

```
cursor.execute("""

        SELECT

            IdSocio,

            Nombre,

            Apellido

        FROM Socios

        WHERE Activo = 1

        ORDER BY Apellido

    """)

resultado = cursor.fetchall()

conexion.close()

return resultado
```

- **Deportes**

```
import tkinter as tk

from tkinter import ttk

from tkinter import messagebox

from modelos.deporte import Deporte
```

```
class VentanaDeportes:

    def __init__(self):

        self.id_seleccionado = None

        self.ventana = tk.Toplevel()

        self.ventana.title(

            "Gestión de Deportes"

        )

        self.ventana.geometry("800x500")

        self.crear_componentes()

        self.cargar_datos()

    def crear_componentes(self):

        frame = tk.Frame(self.ventana)

        frame.pack(pady=10)

        tk.Label(
```



```
        frame,

        text="Nombre"

    ).grid(

        row=0,

        column=0

    )

    self.txt_nombre = tk.Entry(

        frame

    )

    self.txt_nombre.grid(

        row=0,

        column=1

    )

    tk.Label(

        frame,

        text="Cuota"

    ).grid(

        row=0,

        column=2

    )

    self.txt_cuota = tk.Entry(
```

```
        frame

    )

    self.txt_cuota.grid(

        row=0,

        column=3

    )

    tk.Button(

        frame,

        text="Agregar",

        command=self.guardar

    ).grid(

        row=1,

        column=0,

        pady=10

    )

    tk.Button(

        frame,

        text="Modificar",

        command=self.modificar

    ).grid(

        row=1,

        column=1
```

```
)

tk.Button(

    frame,

    text="Eliminar",

    command=self.eliminar

).grid(

    row=1,

    column=2

)

tk.Button(

    frame,

    text="Refrescar",

    command=self.cargar_datos

).grid(

    row=1,

    column=3

)

self.tabla = ttk.Treeview(

    self.ventana,

    columns=(

        "ID",

        "Nombre",
```

```
        "Cuota",

        "Activo"

    ),

    show="headings"

)

self.tabla.heading(

    "ID",

    text="ID"

)

self.tabla.heading(

    "Nombre",

    text="Nombre"

)

self.tabla.heading(

    "Cuota",

    text="Cuota"

)

self.tabla.heading(

    "Activo",

    text="Activo"

)
```

```
self.tabla.pack(

    fill="both",

    expand=True,

    padx=10,

    pady=10

)


self.tabla.bind(

    "<<TreeviewSelect>>",

    self.seleccionar_deporte

)


def cargar_datos(self):

    for fila in self.tabla.get_children():

        self.tabla.delete(fila)

    deportes = Deporte.obtener_todos()

    for deporte in deportes:

        self.tabla.insert(

            "",
```

```
        tk.END,

        values=(

            deporte.IdDeporte,

            deporte.Nombre,

            deporte.CuotaMensual,

            deporte.Activo

        )

    )

def guardar(self):

    Deporte.agregar(

        self.txt_nombre.get(),

        self.txt_cuota.get()

    )

    self.cargar_datos()

    self.limpiar()

    messagebox.showinfo(

        "Correcto",

        "Deporte agregado"

    )
```

```
def seleccionar_deporte(self, event):

    seleccion = self.tabla.selection()

    if not seleccion:

        return

    valores = self.tabla.item(

        seleccion[0]

    )["values"]

    self.id_seleccionado = valores[0]

    self.txt_nombre.delete(

        0,

        tk.END

    )

    self.txt_nombre.insert(

        0,

        valores[1]

    )

    self.txt_cuota.delete(

        0,
```

```
        tk.END

    )

    self.txt_cuota.insert(

        0,

        valores[2]

    )

def modificar(self):

    if self.id_seleccionado is None:

        messagebox.showwarning(

            "Aviso",

            "Seleccione un deporte"

        )

    return

Deporte.actualizar(

    self.id_seleccionado,

    self.txt_nombre.get(),

    self.txt_cuota.get()

)
```



```
self.cargar_datos()

self.limpiar()

messagebox.showinfo(

    "Correcto",

    "Deporte actualizado"

)

def eliminar(self):

    if self.id_seleccionado is None:

        messagebox.showwarning(

            "Aviso",

            "Seleccione un deporte"

        )

        return

    respuesta = messagebox.askyesno(

        "Confirmar",

        "¿Desea desactivar este deporte?"

    )
```

```
if respuesta:

    Deporte.baja_logica(

        self.id_seleccionado

    )

    self.cargar_datos()

    self.limpiar()

    messagebox.showinfo(

        "Correcto",

        "Deporte desactivado"

    )

def limpiar(self):

    self.txt_nombre.delete(

        0,

        tk.END

    )

    self.txt_cuota.delete(

        0,

        tk.END
```

```
)
```

```
self.id_seleccionado = None
```

```
from config.conexion import obtener_conexion
```

```
class Deporte:
```

```
    @staticmethod
```

```
    def obtener_todos():
```

```
        conexion = obtener_conexion()
```

```
        cursor = conexion.cursor()
```

```
        cursor.execute("""
```

```
            SELECT
```

```
                IdDeporte,
```

```
                Nombre,
```

```
                CuotaMensual,
```

```
                Activo
```

```
            FROM Deportes
```

```
            ORDER BY Nombre
```

```
        """)
```

```
deportes = cursor.fetchall()

conexion.close()

return deportes

@staticmethod
def obtener_combo():

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        SELECT

            IdDeporte,

            Nombre

        FROM Deportes

        WHERE Activo = 1

        ORDER BY Nombre

    """)

    resultado = cursor.fetchall()
```

```
        conexion.close()

    return resultado

    @staticmethod
    def agregar(nombre, cuota):

        conexion = obtener_conexion()

        cursor = conexion.cursor()

        cursor.execute("""

            INSERT INTO Deportes

            (

                Nombre,

                CuotaMensual

            )

            VALUES

            (?, ?)

        """,

        (

            nombre,

            cuota

        ))
```

```
        conexion.commit()

    conexion.close()

    @staticmethod
    def actualizar(
        id_deporte,
        nombre,
        cuota
    ):

        conexion = obtener_conexion()

        cursor = conexion.cursor()

        cursor.execute("""
            UPDATE Deportes
            SET
                Nombre = ?,
                CuotaMensual = ?
            WHERE IdDeporte = ?
        """,
        (
            nombre,
            cuota,
```

```
        id_deporte

    ))

    conexion.commit()

    conexion.close()

    @staticmethod
    def baja_logica(id_deporte):

        conexion = obtener_conexion()

        cursor = conexion.cursor()

        cursor.execute("""

            UPDATE Deportes

            SET Activo = 0

            WHERE IdDeporte = ?

        """,

        (

            id_deporte,

        ))

        conexion.commit()
```

```
conexion.close()
```

- **Inscripciones**

```
import tkinter as tk

from tkinter import ttk

from tkinter import messagebox

from modelos.socio import Socio

from modelos.deporte import Deporte

from modelos.inscripcion import Inscripcion


class VentanaInscripciones:

    def __init__(self):

        self.ventana = tk.Toplevel()

        self.ventana.title(

            "Inscripciones Deportivas"

        )

        self.ventana.geometry("800x500")

        self.socios = []
```



```
self.deportes = []

self.crear_componentes()

self.cargar_datos()

def crear_componentes(self):

    frame = tk.Frame(self.ventana)

    frame.pack(pady=10)

    tk.Label(

        frame,

        text="Socio"

    ).grid(row=0, column=0)

    self.cbo_socios = ttk.Combobox(

        frame,

        width=40,

        state="readonly"

    )

    self.cbo_socios.grid(

        row=0,

        column=1,
```

```
        padx=5

    )

    tk.Label(

        frame,

        text="Deporte"

    ).grid(row=1, column=0)


    self.cbo_deportes = ttk.Combobox(

        frame,

        width=40,

        state="readonly"

    )

    self.cbo_deportes.grid(

        row=1,

        column=1,

        padx=5

    )


    tk.Button(

        frame,

        text="Inscribir",

        command=self.guardar

    ).grid(
```

```
        row=2,

        column=0,

        columnspan=2,

        pady=10

    )

    self.tabla = ttk.Treeview(

        self.ventana,

        columns=(

            "ID",

            "Socio",

            "Deporte",

            "Fecha"

        ),

        show="headings"

    )

    self.tabla.heading("ID", text="ID")

    self.tabla.heading("Socio", text="Socio")

    self.tabla.heading("Deporte", text="Deporte")

    self.tabla.heading("Fecha", text="Fecha")

    self.tabla.pack(

        fill="both",

        expand=True,
```

```
        padx=10,

        pady=10

    )

def cargar_datos(self):

    self.socios = Socio.obtener_activos()

    lista_socios = []

    for socio in self.socios:

        lista_socios.append(

            f"{socio.IdSocio} - {socio.Apellido},
{socio.Nombre}"

        )

    self.cbo_socios["values"] = lista_socios

    self.deportes = Deporte.obtener_combo()

    lista_deportes = []

    for deporte in self.deportes:
```

```
        lista_deportes.append(

            f"{deporte.IdDeporte} - {deporte.Nombre}"

        )

    self.cbo_deportes["values"] = lista_deportes

    self.cargar_tabla()

def cargar_tabla(self):

    for fila in self.tabla.get_children():

        self.tabla.delete(fila)

    inscripciones = Inscripcion.obtener_todas()

    for inscripcion in inscripciones:

        self.tabla.insert(

            "",

            tk.END,

            values=(

                inscripcion.IdInscripcion,

                inscripcion.Socio,

                inscripcion.Deporte,
```

```
        inscripcion.FechaInscripcion

    )

)

def guardar(self):

    if not self.cbo_socios.get():

        messagebox.showwarning(

            "Aviso",

            "Seleccione un socio"

        )

    return

    if not self.cbo_deportes.get():

        messagebox.showwarning(

            "Aviso",

            "Seleccione un deporte"

        )

    return

    id_socio = int(
```

```

        self.cbo_socios.get().split("-")[0].strip()

    )

    id_deporte = int(

        self.cbo_deportes.get().split("-")[0].strip()

    )

    Incripcion.agregar(

        id_socio,

        id_deporte

    )

    self.cargar_tabla()

    messagebox.showinfo(

        "Correcto",

        "Inscripción registrada"

    )

```

```

from config.conexion import obtener_conexion

```

```

class Incripcion:

```

```

    @staticmethod

```

```
def agregar(id_socio, id_deporte):
```

```
    conexion = obtener_conexion()
```

```
    cursor = conexion.cursor()
```

```
    cursor.execute("""
```

```
        INSERT INTO Inscripciones
```

```
        (
```

```
            IdSocio,
```

```
            IdDeporte
```

```
        )
```

```
        VALUES
```

```
        (?, ?)
```

```
    """,
```

```
    (
```

```
        id_socio,
```

```
        id_deporte
```

```
    ))
```

```
    conexion.commit()
```

```
    conexion.close()
```

```
@staticmethod
```



```
def obtener_todas():

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        SELECT

            I.IdInscripcion,

            S.Nombre + ' ' + S.Apellido AS Socio,

            D.Nombre AS Deporte,

            I.FechaInscripcion

        FROM Inscripciones I

        INNER JOIN Socios S

            ON I.IdSocio = S.IdSocio

        INNER JOIN Deportes D

            ON I.IdDeporte = D.IdDeporte

        WHERE I.Activo = 1

        ORDER BY S.Apellido

    """)

    resultado = cursor.fetchall()

    conexion.close()
```

```
return resultado
```

- **Pagos**

```
import tkinter as tk

from tkinter import ttk

from tkinter import messagebox

from modelos.socio import Socio

from modelos.pago import Pago


class VentanaPagos:

    def __init__(self):

        self.ventana = tk.Toplevel()

        self.ventana.title(

            "Registro de Pagos"

        )

        self.ventana.geometry("900x500")

        self.crear_componentes()
```

```
self.cargar_socios()

self.cargar_tabla()

def crear_componentes(self):

    frame = tk.Frame(self.ventana)

    frame.pack(pady=10)

    tk.Label(

        frame,

        text="Socio"

    ).grid(row=0, column=0)

    self.cbo_socio = ttk.Combobox(

        frame,

        width=40,

        state="readonly"

    )

    self.cbo_socio.grid(

        row=0,

        column=1

    )
```

```
tk.Label(  
    frame,  
    text="Concepto"  
).grid(row=1, column=0)  
  
self.cbo_concepto = ttk.Combobox(  
    frame,  
    values=[  
        "Cuota Social",  
        "Cuota Deportiva"  
    ],  
    state="readonly"  
)  
  
self.cbo_concepto.grid(  
    row=1,  
    column=1  
)  
  
tk.Label(  
    frame,  
    text="Importe"  
) .grid(row=2, column=0)
```

```
self.txt_importe = tk.Entry(  
  
    frame  
  
)
```

```
self.txt_importe.grid(  
  
    row=2,  
  
    column=1  
  
)
```

```
tk.Button(  
  
    frame,  
  
    text="Registrar Pago",  
  
    command=self.guardar  
  
) .grid(  
  
    row=3,  
  
    column=0,  
  
    columnspan=2,  
  
    pady=10  
  
)
```

```
self.tabla = ttk.Treeview(  
  
    self.ventana,  
  
    columns=(  
  
        "ID",  
  
        "Socio",
```

```
        "Concepto",

        "Importe",

        "Fecha"

    ),

    show="headings"

)

for columna in (

    "ID",

    "Socio",

    "Concepto",

    "Importe",

    "Fecha"

):

    self.tabla.heading(

        columna,

        text=columna

    )

self.tabla.pack(

    fill="both",

    expand=True

)

def cargar_socios(self):
```

```
socios = Socio.obtener_activos()

self.cbo_socio["values"] = [

    f"{s.IdSocio} - {s.Apellido}, {s.Nombre}"

    for s in socios

]

def cargar_tabla(self):

    for fila in self.tabla.get_children():

        self.tabla.delete(fila)

    pagos = Pago.obtener_todos()

    for pago in pagos:

        self.tabla.insert(

            "",

            tk.END,

            values=(

                pago.IdPago,

                pago.Socio,

                pago.Concepto,
```

```
                pago.Importe,

                pago.FechaPago

            )

        )

def guardar(self):

    if not self.cbo_socio.get():

        return

    id_socio = int(

        self.cbo_socio.get().split("-")[0]

    )

    Pago.registrar(

        id_socio,

        self.cbo_concepto.get(),

        self.txt_importe.get()

    )

    self.cargar_tabla()

    messagebox.showinfo(

        "Comprobante de Pago",
```



```

        f"""

        CLUB VILLA DEL PARQUE II

        Socio:

        {self.cbo_socio.get()}

        Concepto:

        {self.cbo_concepto.get()}

        Importe:

        ${self.txt_importe.get()}

        Estado:

        PAGO REGISTRADO

        """

    )

```

```

from config.conexion import obtener_conexion

class Pago:

    @staticmethod

    def registrar(id_socio, concepto, importe):

```

```
conexion = obtener_conexion()
```

```
cursor = conexion.cursor()
```

```
cursor.execute("""
```

```
    INSERT INTO Pagos
```

```
    (
```

```
        IdSocio,
```

```
        Concepto,
```

```
        Importe
```

```
    )
```

```
VALUES
```

```
    (?, ?, ?)
```

```
""",
```

```
(
```

```
    id_socio,
```

```
    concepto,
```

```
    importe
```

```
))
```

```
conexion.commit()
```

```
conexion.close()
```

```
@staticmethod
```

```
def obtener_todos():

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        SELECT

            P.IdPago,

            S.Nombre + ' ' + S.Apellido AS Socio,

            P.Concepto,

            P.Importe,

            P.FechaPago

        FROM Pagos P

        INNER JOIN Socios S

            ON P.IdSocio = S.IdSocio

        ORDER BY P.FechaPago DESC

    """)

    datos = cursor.fetchall()

    conexion.close()

    return datos

@staticmethod
```

```
def total_recaudado():

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        SELECT ISNULL(

            SUM(Importe),

            0

        )

        FROM Pagos

    """)

    total = cursor.fetchone()[0]

    conexion.close()

    return total

@staticmethod

def recaudacion_por_concepto():

    conexion = obtener_conexion()

    cursor = conexion.cursor()
```

```
cursor.execute("""

        SELECT

            Concepto,

            SUM(Importe) AS Total

        FROM Pagos

        GROUP BY Concepto

    """)

datos = cursor.fetchall()

conexion.close()

return datos

@staticmethod
def pagos_mes_actual():

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        SELECT

            COUNT(*)

        FROM Pagos

    """)
```

```

        WHERE

            MONTH(FechaPago)=MONTH(GETDATE())

        AND

            YEAR(FechaPago)=YEAR(GETDATE())

        """)

    cantidad = cursor.fetchone()[0]

    conexion.close()

    return cantidad

```

- **Invitados**

```

import tkinter as tk

from tkinter import ttk

from tkinter import messagebox

from modelos.invitado import Invitado

from modelos.deporte import Deporte


class VentanaInvitados:

    def __init__(self):

```

```
self.ventana = tk.Toplevel()

self.ventana.title(

    "Gestión de Invitados"

)

self.ventana.geometry(

    "900x500"

)

self.crear_componentes()

self.cargar_deportes()

self.cargar_tabla()

def crear_componentes(self):

    frame = tk.Frame(self.ventana)

    frame.pack(pady=10)

    tk.Label(frame, text="Nombre").grid(row=0, column=0)

    self.txt_nombre = tk.Entry(frame)

    self.txt_nombre.grid(row=0, column=1)
```

```
tk.Label(frame, text="Apellido").grid(row=0, column=2)

self.txt_apellido = tk.Entry(frame)

self.txt_apellido.grid(row=0, column=3)


tk.Label(frame, text="DNI").grid(row=1, column=0)

self.txt_dni = tk.Entry(frame)

self.txt_dni.grid(row=1, column=1)


tk.Label(frame, text="Importe").grid(row=1, column=2)

self.txt_importe = tk.Entry(frame)

self.txt_importe.grid(row=1, column=3)


tk.Label(frame, text="Deporte").grid(row=2, column=0)


self.cbo_deporte = ttk.Combobox(

    frame,

    state="readonly"

)


self.cbo_deporte.grid(

    row=2,

    column=1

)
```



```
tk.Button(  
  
    frame,  
  
    text="Registrar Invitado",  
  
    command=self.guardar  
  
).grid(  
  
    row=3,  
  
    column=0,  
  
    columnspan=4,  
  
    pady=10  
  
)
```

```
self.tabla = ttk.Treeview(  
  
    self.ventana,  
  
    columns=(  
  
        "ID",  
  
        "Invitado",  
  
        "DNI",  
  
        "Deporte",  
  
        "Importe",  
  
        "Fecha"  
  
    ),  
  
    show="headings"  
  
)
```

```
for column in (
```

```

        "ID",

        "Invitado",

        "DNI",

        "Deporte",

        "Importe",

        "Fecha"

    ):

        self.tabla.heading(

            columnna,

            text=columnna

        )

self.tabla.pack(

    fill="both",

    expand=True

)

def cargar_deportes(self):

    self.deportes = Deporte.obtener_combo()

    self.cbo_deporte["values"] = [

        f"{d.IdDeporte} - {d.Nombre}"

        for d in self.deportes

    ]

```

```
def cargar_tabla(self):

    for fila in self.tabla.get_children():

        self.tabla.delete(fila)

    invitados = Invitado.obtener_todos()

    for invitado in invitados:

        self.tabla.insert(

            "",

            tk.END,

            values=(

                invitado.IdInvitado,

                invitado.Invitado,

                invitado.DNI,

                invitado.Deporte,

                invitado.Importe,

                invitado.FechaVisita

            )

        )

def guardar(self):
```

```
id_deporte = int(

    self.cbo_deporte.get().split("-")[0]

)

Invitado.agregar(

    self.txt_nombre.get(),

    self.txt_apellido.get(),

    self.txt_dni.get(),

    id_deporte,

    self.txt_importe.get()

)

self.cargar_tabla()

messagebox.showinfo(

    "Correcto",

    "Invitado registrado"

)
```

```
from config.conexion import obtener_conexion

class Invitado:
```

```
@staticmethod

def agregar(

    nombre,

    apellido,

    dni,

    id_deporte,

    importe

):

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        INSERT INTO Invitados

        (

            Nombre,

            Apellido,

            DNI,

            IdDeporte,

            Importe

        )

        VALUES

        (?, ?, ?, ?, ?)

    """,
```

```
(
    nombre,

    apellido,

    dni,

    id_deporte,

    importe
))

conexion.commit()

conexion.close()

@staticmethod
def obtener_todos():

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        SELECT

            I.IdInvitado,

            I.Nombre + ' ' + I.Apellido AS Invitado,

            I.DNI,

            D.Nombre AS Deporte,
```

```

        I.Importe,

        I.FechaVisita

    FROM Invitados I

    INNER JOIN Deportes D

        ON I.IdDeporte = D.IdDeporte

    ORDER BY I.FechaVisita DESC

    """

    datos = cursor.fetchall()

    conexion.close()

    return datos

```

- **Reportes**

```

import tkinter as tk

from modelos.pago import Pago

from modelos.reportes import Reporte

class VentanaReportes:

```

```
def __init__(self):

    self.ventana = tk.Toplevel()

    self.ventana.title(

        "Reportes y Estadísticas"

    )

    self.ventana.geometry(

        "700x650"

    )

    self.cargar_datos()

def cargar_datos(self):

    total = Reporte.total_recaudado()

    total_invitados = Reporte.total_invitados()

    cantidad_socios = Reporte.cantidad_socios()

    cantidad_inscripciones = Reporte.cantidad_inscripciones()

    cantidad_invitados = Reporte.cantidad_invitados()
```



```
pagos_mes = Reporte.pagos_mes_actual()

deporte_popular = Reporte.deporte_mas_popular()

tk.Label(

    self.ventana,

    text="CLUB VILLA DEL PARQUE II",

    font=("Arial", 18, "bold")

).pack(pady=10)

tk.Label(

    self.ventana,

    text="RENDICIÓN Y ESTADÍSTICAS",

    font=("Arial", 14)

).pack(pady=5)

tk.Label(

    self.ventana,

    text=f"Socios Activos: {cantidad_socios}"

).pack(pady=3)

tk.Label(

    self.ventana,
```

```
        text=f"Inscripciones Deportivas:
{cantidad_inscripciones}"

        ).pack(pady=3)

    tk.Label(

        self.ventana,

        text=f"Invitados Registrados: {cantidad_invitados}"

    ).pack(pady=3)

    tk.Label(

        self.ventana,

        text=f"Pagos Registrados Este Mes: {pagos_mes}"

    ).pack(pady=3)

    tk.Label(

        self.ventana,

        text=f"Total Recaudado por Socios: ${total}"

    ).pack(pady=3)

    tk.Label(

        self.ventana,

        text=f"Total Recaudado por Invitados:
${total_invitados}"

    ).pack(pady=3)

    if deporte_popular:
```

```
        tk.Label(

            self.ventana,

            text=f"Deporte Más Popular: {deporte_popular[0]}
({deporte_popular[1]} inscripciones)"

        ).pack(pady=10)


    tk.Label(

        self.ventana,

        text="RECAUDACIÓN POR CONCEPTO",

        font=("Arial", 12, "bold")

    ).pack(pady=10)


    conceptos = Reporte.recaudacion_por_concepto()


    for concepto in conceptos:

        tk.Label(

            self.ventana,

            text=f"{concepto[0]}: ${concepto[1]}"

        ).pack()


    tk.Label(

        self.ventana,

        text="SOCIOS POR DEPORTE",
```

```
        font=("Arial", 12, "bold")

    ).pack(pady=15)

    deportes = Reporte.socios_por_deporte()

    for deporte in deportes:

        tk.Label(

            self.ventana,

            text=f"{deporte[0]}: {deporte[1]} socios"

        ).pack()

    tk.Label(

        self.ventana,

        text="INVITADOS POR DEPORTE",

        font=("Arial", 12, "bold")

    ).pack(pady=15)

    invitados = Reporte.invitados_por_deporte()

    for invitado in invitados:

        tk.Label(

            self.ventana,

            text=f"{invitado[0]}: {invitado[1]} invitados"
```

```
).pack()
```

```
from config.conexion import obtener_conexion
```

```
class Reporte:
```

```
    @staticmethod
```

```
    def cantidad_socios():
```

```
        conexion = obtener_conexion()
```

```
        cursor = conexion.cursor()
```

```
        cursor.execute("""
```

```
            SELECT COUNT(*)
```

```
            FROM Socios
```

```
            WHERE Activo = 1
```

```
        """)
```

```
        resultado = cursor.fetchone()[0]
```

```
        conexion.close()
```

```
        return resultado
```

```
@staticmethod

def cantidad_inscripciones():

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        SELECT COUNT(*)

        FROM Inscripciones

        WHERE Activo = 1

    """)

    resultado = cursor.fetchone()[0]

    conexion.close()

    return resultado

@staticmethod

def total_recaudado():

    conexion = obtener_conexion()
```

```
cursor = conexion.cursor()

cursor.execute("""

    SELECT ISNULL(

        SUM(Importe),

        0

    )

    FROM Pagos

""")

total = cursor.fetchone()[0]

conexion.close()

return total

@staticmethod
def total_invitados():

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        SELECT ISNULL(
```

```
        SUM(Importe),

        0

    )

    FROM Invitados

    """

total = cursor.fetchone()[0]

conexion.close()

return total

@staticmethod
def socios_por_deporte():

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        SELECT

            D.Nombre,

            COUNT(*) AS Cantidad

        FROM Inscripciones I

        INNER JOIN Deportes D
```



```
        ON I.IdDeporte = D.IdDeporte

        WHERE I.Activo = 1

        GROUP BY D.Nombre

        ORDER BY Cantidad DESC

    """

    datos = cursor.fetchall()

    conexion.close()

    return datos

    @staticmethod
    def deporte_mas_popular():

        conexion = obtener_conexion()

        cursor = conexion.cursor()

        cursor.execute("""

            SELECT TOP 1

                D.Nombre,

                COUNT(*) AS Cantidad

            FROM Inscripciones I

            INNER JOIN Deportes D
```

```
        ON I.IdDeporte = D.IdDeporte

        WHERE I.Activo = 1

        GROUP BY D.Nombre

        ORDER BY Cantidad DESC

    """

    resultado = cursor.fetchone()

    conexion.close()

    return resultado

    @staticmethod
    def recaudacion_por_concepto():

        conexion = obtener_conexion()

        cursor = conexion.cursor()

        cursor.execute("""

            SELECT

                Concepto,

                SUM(Importe) AS Total

            FROM Pagos

            GROUP BY Concepto
```

```
ORDER BY Concepto

"""

datos = cursor.fetchall()

conexion.close()

return datos

@staticmethod
def pagos_mes_actual():

    conexion = obtener_conexion()

    cursor = conexion.cursor()

    cursor.execute("""

        SELECT COUNT(*)

        FROM Pagos

        WHERE

            MONTH(FechaPago) = MONTH(GETDATE())

        AND

            YEAR(FechaPago) = YEAR(GETDATE())

    """)
```

```
        resultado = cursor.fetchone()[0]

    conexion.close()

    return resultado

    @staticmethod
    def ultimos_pagos():

        conexion = obtener_conexion()

        cursor = conexion.cursor()

        cursor.execute("""

            SELECT TOP 10

                IdPago,

                Concepto,

                Importe,

                FechaPago

            FROM Pagos

            ORDER BY FechaPago DESC

        """)

        datos = cursor.fetchall()
```

```
        conexion.close()

    return datos

    @staticmethod
    def invitados_por_deporte():

        conexion = obtener_conexion()

        cursor = conexion.cursor()

        cursor.execute("""

            SELECT

                D.Nombre,

                COUNT(*) AS Cantidad

            FROM Invitados I

            INNER JOIN Deportes D

                ON I.IdDeporte = D.IdDeporte

            GROUP BY D.Nombre

            ORDER BY Cantidad DESC

        """)

        datos = cursor.fetchall()

        conexion.close()
```

```
        return datos

    @staticmethod
    def cantidad_invitados():

        conexion = obtener_conexion()

        cursor = conexion.cursor()

        cursor.execute("""

            SELECT COUNT(*)

            FROM Invitados

        """)

        resultado = cursor.fetchone()[0]

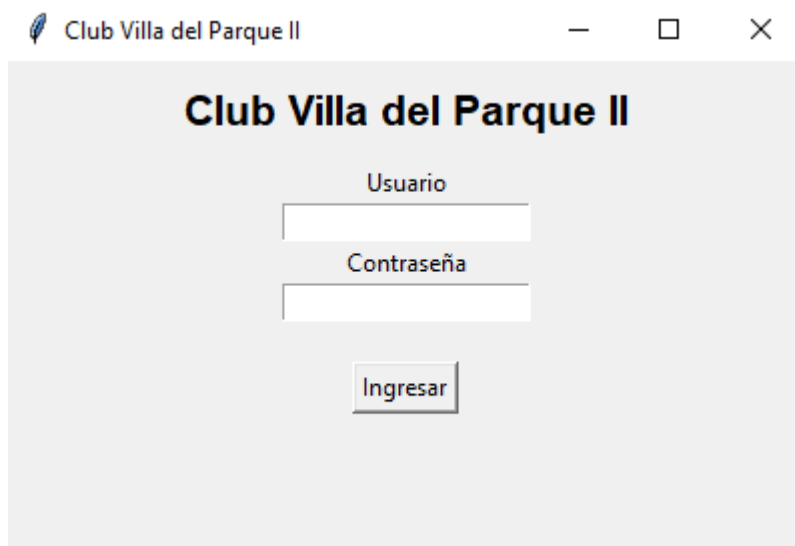
        conexion.close()

        return resultado
```

Anexo C – Capturas de Pantalla

Se incorporan capturas de:

- Login



The screenshot shows a web browser window titled "Club Villa del Parque II". The page has a light gray background. At the top, the title "Club Villa del Parque II" is displayed in a bold, black font. Below the title, there are two input fields: "Usuario" (Username) and "Contraseña" (Password). Below these fields is a button labeled "Ingresar" (Login).

- Dashboard



The screenshot shows a web browser window titled "Dashboard - Club Villa del Parque II". The page has a light gray background. At the top, the title "CLUB VILLA DEL PARQUE II" is displayed in a bold, black font. Below the title, the user's login information is shown: "Usuario: admin" and "Rol: Administrador". Below this information, there is a vertical list of buttons: "Gestión de Socios", "Gestión de Deportes", "Inscripciones", "Pagos", "Invitados", "Reportes", and "Salir".

- Gestión de Socios

Gestión de Socios

Nombre

Apellido

DNI

Teléfono

Dirección

Guardar Socio

Modificar

Eliminar

Refrescar

ID	Nombre	Apellido	DNI	Teléfono	Dirección	Activo
5	Pepe	Argento	18000333	11333332	Flores 223	True

- Gestión de Deportes

Gestión de Deportes

Nombre

Cuota

Agregar

Modificar

Eliminar

Refrescar

ID	Nombre	Cuota	Activo
2	Básquet	15000.00	True
1	Fútbol	12000.00	True
4	Taekwondo	35000.00	True
3	Tenis	18000.00	True

- Inscripciones

Inscripciones Deportivas

Socio5 - Argento, Pepe

Deporte4 - Taekwondo

Inscribir

ID	Socio	Deporte	Fecha
2	Pepe Argento	Taekwondo	2026-06-18

- Pagos

Registro de Pagos

Socio5 - Argento, Pepe

ConceptoCuota Deportiva

Importe35000

Registrar Pago

ID	Socio	Concepto	Importe	Fecha
2	Pepe Argento	Cuota Deportiva	35000.00	2026-06-18 12:11:25

- Invitados

Gestión de Invitados

Nombre

Pepe

Apellido

Grillo

DNI

121321313

Importe

12000

Deporte

1 - Fútbol

Registrar Invitado

ID	Invitado	DNI	Deporte	Imp
1	Pepe Grillo	121321313	Fútbol	12000.00

- Reportes

Reportes y Estadísticas

CLUB VILLA DEL PARQUE II

RENDICIÓN Y ESTADÍSTICAS

Socios Activos: 1

Inscripciones Deportivas: 1

Invitados Registrados: 1

Pagos Registrados Este Mes: 2

Total Recaudado por Socios: \$70000.00

Total Recaudado por Invitados: \$12000.00

Deporte Más Popular: Taekwondo (1 inscripciones)

RECAUDACIÓN POR CONCEPTO

Cuota Deportiva: \$70000.00

SOCIOS POR DEPORTE

Taekwondo: 1 socios

INVITADOS POR DEPORTE

Fútbol: 1 invitados

20. BIBLIOGRAFÍA

Python Software Foundation

[Python Documentation](#)

Documentación oficial del lenguaje Python utilizada durante el desarrollo.

Microsoft

[Microsoft SQL Server Documentation](#)

Documentación oficial de SQL Server.

Tkinter

[Tkinter Documentation](#)

Documentación oficial de la biblioteca gráfica utilizada.

PyODBC

[PyODBC Documentation](#)

Documentación utilizada para la conexión con SQL Server.

CONCLUSIÓN GENERAL DEL TRABAJO

El Sistema de Gestión Deportiva Club Villa del Parque II constituye una solución integral para la administración de entidades deportivas de pequeña y mediana escala.

La implementación permitió integrar conocimientos de:

- Programación orientada a objetos.
- Desarrollo de interfaces gráficas.
- Bases de datos relacionales.

- SQL Server.
- Python.
- Diseño de sistemas de información.

El resultado obtenido es una aplicación funcional capaz de gestionar socios, deportes, inscripciones, pagos, invitados y reportes administrativos, cumpliendo con los objetivos planteados al inicio del proyecto y proporcionando una base sólida para futuras ampliaciones y mejoras.